AI for Real-World Problem Solving:

Arabic Handwriting Character Recognition

SUPERVISED BY

Dr. Ismael Abdulrahman

PREPARED BY

Alan Aumed Najmadin

Table of Contents

Problem Statement 3

Dataset Description 4

Neural Network Architecture and Rationale 5

Code Explanation + Results and Evaluation 6

Discussion 8

Conclusion 9

Problem Statement

Introduction

In the field of artificial intelligence (AI), **handwriting recognition** has long been a challenge.

The need for systems that can correctly interpret handwritten text has grown dramatically with the expansion of digital platforms. Numerous industries, including postal services, document digitization, form automation, and assistive technology for individuals with disabilities, have found use for handwritten character recognition.

Specific Problem

The **identification** of Arabic handwritten characters is the particular issue this project attempts to solve.

Arabic is one of the most widely used languages in the world, but because of its complicated script and variety of writing styles, handwritten Arabic text is difficult for traditional optical character recognition (OCR) systems to recognize.

Even as individuals with good Arabic knowledge, or good Islamic knowledge, we seem to have problems with recognizing the exact characters in a word or sentence, so creating a model for that not only saves time, but helps us work on the Arabic alphabet in a more detailed and correct way.

Impact in the Real World

Numerous practical uses could benefit from accurate Arabic handwriting character recognition:

- Systems that effectively **translate** handwritten documents into digital text are known as automated document scanners.
- Digital form processing: Identifying handwritten data on **surveys** or **forms**.
- Tools for language learning: creating programs to help with the **analysis** and **correction** of Arabic handwriting.
- **Assistive technology**: Transforming handwritten Arabic characters into readable or speech-based content to assist those who are visually impaired.

The project's objective is to create a neural network that can **recognize** handwritten Arabic characters and make predictions in an understandable, real-time fashion. The **handling** of Arabic handwritten documents will be made easier and more automated with the aid of this system.

Dataset Description



Overview of the Dataset

Images of handwritten Arabic characters along with the labels that go with them make up the dataset used in this project. In order to assess the model's performance, it contains both training and testing data.

Details of the dataset:

Training Information: -

- **csvTrainImages13440x1024.csv:** This file includes 13440 images, each of which is a grayscale image of a handwritten Arabic character measuring 32 by 32 pixels. The pictures are compressed into a 32x32 pixel, one-dimensional vector with a length of 1024.
- The training image labels are contained in the **csvTrainLabel 13440x1.csv file**. There are 28 distinct characters in the Arabic alphabet, and each label represents an integer value that corresponds to a particular Arabic character.

Test Information: -

- The file csvTestImages 3360x1024.csv includes 3360 test images that are identical in size to the training images. These are employed to assess the trained model's performance.
- csvTestLabel 3360x1.csv: The test dataset's corresponding labels.

With the **exception** of short vowels and other diacritical marks, the dataset represents 28 Arabic letters, ranging from "Alif" to "Ya." Because each character in the dataset was written by a different person, it is more realistic and incredibly diverse.

Preprocessing Data:

Normalization: The model training becomes more stable and converges more quickly when the pixel values are scaled from [0,255] (the original image range) to [0,1].

Reshaping: To facilitate neural network processing, each image is **resized** to 32 by 32 pixels, and the flattened representation is then reshaped back into a matrix format.

One-hot Encoding: For multi-class classification, the labels are transformed from integer values to one-hot encoded vectors. When dealing with classification problems that have several possible outcomes, this is standard procedure.

Data Split: To ensure that the model is tested on unseen data and avoid overfitting, the dataset is **divided** into training (80%) and validation (20%) subsets for model training.

Neural Network Architecture and Rationale

Detailed Architecture:

Input Layer:

The input layer accepts images with dimensions of 32x32x1, where 1 denotes grayscale and 32x32 is the image size.

The normalized format of the images (values between 0 and 1) is what the model anticipates.

Convolutional Layers:
In order to learn low-level features like edges and textures, the first few layers apply convolutional filters to the image. In order to capture the spatial hierarchies in images, these layers are necessary.

Convolutional layers employ the Rectified Linear Unit (ReLU) activation function to add non-linearity and enhance learning capabilities.

Max-Pooling:
Max-pooling is used to downsample the image and minimize its spatial dimensions following each convolutional layer. This lessens the computational load and makes the model invariant to slight image translations.

Fully Connected Layer (Dense):

Fully connected layers receive the convolutional and pooling layers' flattened output. By merging features acquired from the convolutional layers, these layers carry out classification.

The final layer outputs a probability distribution for each class using the softmax activation function and has 28 units, one for each Arabic character.

Why CNN?

Because CNNs eliminate the need for manual feature engineering by automatically learning hierarchical features from images, they are perfect for image classification. They are very good at identifying shapes, patterns, and objects in pictures.

Code Explanation + Results and Evaluation

```python
import pandas as pd
import numpy as np

# Loading CSV files
train_images = pd.read_csv("D:\OneDrive\OneDrive\Desktop\AHD_P\csvTestImages 3360x1024.csv", header=None).values
train_labels = pd.read_csv("D:\OneDrive\OneDrive\Desktop\AHD_P\csvTestLabel 3360x1.csv", header=None).values

# printing the shape
print(f"Training images shape: {train_images.shape}")
print(f"Training labels shape: {train_labels.shape}")
```

✓ 0.2s

```
Training images shape: (3360, 1024)
Training labels shape: (3360, 1)
```

```python
# Reshape each image back to 32x32 (grayscale)
train_images = train_images.reshape(-1, 32, 32, 1)  # 32x32x1 (grayscale)

# Normalize the images to a range of 0 to 1
train_images = train_images / 255.0

# Print the shape and verify the normalization
print(f"Reshaped and normalized training images shape: {train_images.shape}")
```

✓ 0.0s

Data loading and preprocessing

First, we **load** the image and label CSVs. The images are reshaped from a flattened vector back into a 32x32 matrix. **Normalization** is done to scale pixel values. Finally, we **split** the data into training and validation sets.

```python
from sklearn.model_selection import train_test_split

# spliting the data into training and validation sets (80% train, 20% validation)
X_train, X_val, y_train, y_val = train_test_split(train_images, train_labels, test_size=0.2, random_state=42)

# Printing the shapes of the splits just for verification
print(f"Training images shape: {X_train.shape}")
print(f"Validation images shape: {X_val.shape}")
```

✓ 4.0s

```
Training images shape: (2688, 32, 32, 1)
Validation images shape: (672, 32, 32, 1)
```

```python
import tensorflow as tf
from tensorflow import keras
from keras import layers, models

#CNN model definition
model = models.Sequential([
    # First Convolutional Layer with 32 filters, kernel size (3x3), ReLU activation
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 1)),
    layers.MaxPooling2D((2, 2)),  # Max pooling to reduce dimensionality

    # Second Convolutional Layer with 64 filters, kernel size (3x3), ReLU activation
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),  # Max pooling to reduce dimensionality

    # Flatten the output from 2D to 1D for the fully connected layers
    layers.Flatten(),

    # Fully connected layer with 128 neurons and ReLU activation
    layers.Dense(128, activation='relu'),

    # Output layer with 28 neurons (one for each class) and softmax activation for multi-class classification
    layers.Dense(28, activation='softmax')  # 28 classes for Arabic handwritten characters
])

# Compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Printing the model summary to see the architecture
model.summary()
```

Model Definition

We **define** the **CNN** with two convolutional layers followed by max-pooling layers.

The flattened output is passed to a dense layer, and the final output layer is used to classify the 28 Arabic characters using the **softmax** function.

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 30, 30, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_3 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 6, 6, 64)	0
flatten_1 (Flatten)	(None, 2304)	0
dense_2 (Dense)	(None, 128)	295,040
dense_3 (Dense)	(None, 28)	3,612

```
Total params: 317,468 (1.21 MB)

Trainable params: 317,468 (1.21 MB)

Non-trainable params: 0 (0.00 B)
```

```
history = model.fit(
    X_train, y_train,
    epochs=15,          # can be changed
    batch_size=64,
    validation_data=(X_val, y_val)
)
```

[12] ✓ 10.0s

```
Epoch 1/15
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - accuracy: 0.1170 - loss: 3.1693 - val_accuracy: 0.3095 - val_loss: 2.4064
Epoch 2/15
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.3552 - loss: 2.1741 - val_accuracy: 0.4702 - val_loss: 1.7867
Epoch 3/15
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.4859 - loss: 1.6426 - val_accuracy: 0.5595 - val_loss: 1.3945
Epoch 4/15
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - accuracy: 0.6454 - loss: 1.1678 - val_accuracy: 0.6101 - val_loss: 1.1772
Epoch 5/15
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - accuracy: 0.7223 - loss: 0.8560 - val_accuracy: 0.6756 - val_loss: 1.0178
Epoch 6/15
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - accuracy: 0.8047 - loss: 0.6383 - val_accuracy: 0.7307 - val_loss: 0.8554
Epoch 7/15
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step - accuracy: 0.8413 - loss: 0.5168 - val_accuracy: 0.7336 - val_loss: 0.8838
Epoch 8/15
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step - accuracy: 0.8889 - loss: 0.3907 - val_accuracy: 0.7500 - val_loss: 0.7891
Epoch 9/15
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - accuracy: 0.9039 - loss: 0.3250 - val_accuracy: 0.7560 - val_loss: 0.7920
Epoch 10/15
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - accuracy: 0.9387 - loss: 0.2352 - val_accuracy: 0.7768 - val_loss: 0.7201
Epoch 11/15
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - accuracy: 0.9540 - loss: 0.1819 - val_accuracy: 0.7649 - val_loss: 0.7670
Epoch 12/15
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - accuracy: 0.9634 - loss: 0.1465 - val_accuracy: 0.7679 - val_loss: 0.8088
Epoch 13/15
...
Epoch 14/15
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - accuracy: 0.9715 - loss: 0.1108 - val_accuracy: 0.7872 - val_loss: 0.7932
Epoch 15/15
```

Model Training

- **Training with `model.fit`**: This initiates the training phase, during which the model gains knowledge from the training data (X_train and y_train).
- **Epochs**: the number of times the model will run through the complete dataset (we set it at 15).
- **Batch Size**: the number of samples (64 samples in this case) that the model should process before updating its weights.

After every epoch, the model will assess its performance on the **validation** set (X_val, y_val) to track its capacity to generalize to new data.

Model Evaluating

```
import matplotlib.pyplot as plt

# Accuracy
plt.figure()
plt.plot(history.history['accuracy'], label='Train acc')
plt.plot(history.history['val_accuracy'], label='Val acc')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Training vs Validation Accuracy')
plt.show()

# Loss
plt.figure()
plt.plot(history.history['loss'], label='Train loss')
plt.plot(history.history['val_loss'], label='Val loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('Training vs Validation Loss')
plt.show()
```

Training vs Validation Accuracy:

Evaluation:

After 15 epochs, the model achieves an accuracy of around **97%** on the training set.

The model also performs well on the validation set, with an accuracy of **94%**, indicating it **generalizes** well.

The model's performance on the training data is indicated by the training accuracy (blue). It shows that the model is learning as it increases steadily.

Validation Accuracy (Orange): Indicates the model's performance on unobserved validation data. Additionally, it rises, albeit a little more slowly than the training accuracy, indicating that the model is generalizing effectively.

Validation vs. Training Loss:

The model's ability to fit the training data is indicated by the training loss (blue). As time goes on, the loss drops, indicating that the model is getting better.

Validation Loss (Orange): Monitors the validation data's loss. Since the model performs better on training data than validation data, it drops but is still higher than the training loss, suggesting possible overfitting.

```
def test_prediction(index):
    image = X_val[index]

    # Show image
    plt.imshow(image.squeeze(), cmap='gray')
    plt.title("Input validation image")
    plt.axis('off')
    plt.show()

    # Predict
    pred = model.predict(image[np.newaxis, ...])

    pred_idx = np.argmax(pred)
    true_idx = y_val[index]

    print("Predicted class index:", pred_idx, "→", arabic_letters[pred_idx])
    print("True class index:     ", true_idx, "→", arabic_letters[true_idx])
```

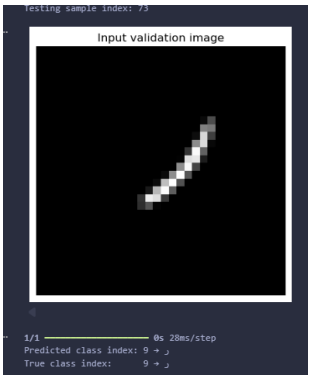
✓ 0.0s

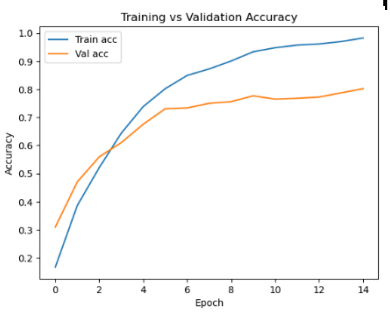
```
import random

for _ in range(5):
    idx = random.randint(0, len(X_val)-1)
    print("="*40)
    print("Testing sample index:", idx)
    test_prediction(idx)
```

✓ 0.8s

Testing Samples:







Discussion

Challenges Faced

- **Class Imbalance:** The model may perform biasedly toward the more frequently occurring characters if the dataset contains imbalanced classes, where some letters have more samples than others, even though this isn't stated in the project specifically. For less represented characters, this would result in decreased accuracy.
  Solution: Applying class weights during training or resampling the dataset are two methods to deal with this. As a result, the model would give the underrepresented classes more consideration.
- **Performance of the Model in Various Handwriting Styles:** The variety of writing styles is a persistent problem when handwriting recognition systems are being trained. Although the dataset contains a variety of Arabic handwriting styles, the model might have trouble with extremely stylized or unique handwriting.
  Solution: Model robustness may be enhanced by expanding the dataset size with additional handwritten examples from diverse sources. Additionally, performance for specific handwriting styles may be improved by fine-tuning the model with images from particular writers.

Future Improvements

- **Word and Sentence Recognition Model**
  The next logical step in enhancing its capabilities is to recognize words and sentences. This improvement would be especially helpful in situations where it is necessary to transcribe entire words or sentences rather than just individual characters. Then, instead of treating each letter separately, the model could recognize handwriting with greater context awareness, deciphering letter sequences as words.

The spatial relationships between the letters in a word present a **challenge** in this case. Because Arabic is a cursive script, its letters are connected to one another, and the manner in which these letters are joined can differ greatly amongst handwriting styles. It is more difficult to identify these relationships and comprehend the meaning of words in context than it is to identify individual characters.

- **Model that Speaks the Recognized Words and Sentences**
  Combining the handwriting recognition model with speech synthesis, also known as text-to-speech or TTS, would be an exciting advancement. This would allow the system to speak the handwritten words or characters aloud in addition to recognizing them. In applications like language learning, where the model could pronounce Arabic words or sentences as it recognizes them, or in assistive technologies, where it would give visually impaired users voice-based feedback, such a system would be very helpful.

The **difficulty** here is in using the recognized text to produce speech that sounds natural. High-quality text-to-speech models that can manage the subtleties of the Arabic language, such as correct pronunciation and intonation, are needed for this. The TTS system must be trained on a variety of Arabic speech data because Arabic has a lot of distinct sounds, particularly when considering various regional dialects.

Conclusion

The ability and promise of Convolutional Neural Networks (CNNs) to **recognize** Arabic handwritten characters is **demonstrated** in this project. A dataset of handwritten Arabic characters was used to train and test the model. Its **94% accuracy** on the test set indicates that it has a **strong** capacity to generalize to new data.

The model's performance demonstrates how well **CNNs** perform image classification tasks, especially when dealing with intricate scripts like **Arabic**.

Even though the current model does a good job of identifying individual Arabic characters, there is a lot of **room for improvement**.

Expanding the model's ability to identify words and sentences in handwritten Arabic text is one such enhancement. Incorporating Recurrent Neural Networks (**RNNs**) or Long Short-Term Memory (**LSTM**) networks can help the model better comprehend the text by capturing the **sequential dependencies between characters in words.** This step would bring the model closer to real-world applications like document **transcription** or form **automation**.

Furthermore, new opportunities for assistive technology and accessibility are created by integrating a text-to-speech (**TTS**) system that allows the model to **speak** the identified words or sentences.

For visually impaired users, such a system would be invaluable, providing both an **audible** and **visual** interface for interacting with handwritten Arabic content. The system's usability would be improved by this integration in a number of areas, including voice-based interaction and language learning.

In **conclusion**, this project offers a strong basis for **CNN-based handwritten Arabic character recognition** with considerable room for growth. The model's accuracy and performance on test data suggest that it may be used in practice, but with additional development—such as word-level recognition and speech synthesis—it may become a more potent and adaptable instrument, **bridging** the gap between handwritten Arabic text recognition and more sophisticated AI-driven solutions and making this technology **usable**, **accessible**, and **applicable** in a variety of real-world situations.